

Moosic — Quantitative Scalability Analysis

1. Purpose

This section evaluates how the Moosic digital business system would behave as the registered user base and peak traffic increase. The calculations follow the CIA III assumptions and are used to explain the scaling implications for application capacity, network traffic, database load, and infrastructure.

2. User Growth Projection

Assumption

Initial registered users = 10,000
Annual growth rate = 25%

Formula

Users after n years = Initial Users × (1 + Growth Rate)ⁿ

Calculations

Year	Formula	Calculation	Expected Users
Year 1	$10,000 \times 1.25^1$	$10,000 \times 1.25$	12,500
Year 2	$10,000 \times 1.25^2$	$10,000 \times 1.5625$	15,625
Year 3	$10,000 \times 1.25^3$	$10,000 \times 1.953125$	$19,531.25 \approx 19,531$
Year 4	$10,000 \times 1.25^4$	$10,000 \times 2.44140625$	$24,414.06 \approx 24,414$
Year 5	$10,000 \times 1.25^5$	$10,000 \times 3.0517578125$	$30,517.58 \approx 30,518$

Interpretation

At the assumed 25% annual growth rate, Moosic grows from 10,000 registered users to approximately 30,518 users after five years. This indicates sustained growth and supports the need for an architecture that can increase application, database, storage, and network capacity without redesigning the complete system.

3. Peak Concurrent Users

Assumption

10% of registered users are active simultaneously during peak periods.

Formula

Peak Concurrent Users = Registered Users × 10%

Registered Users	Formula	Calculation	Peak Concurrent Users
100,000	Users × 10%	$100,000 \times 0.10$	10,000
500,000	Users × 10%	$500,000 \times 0.10$	50,000
1,000,000	Users × 10%	$1,000,000 \times 0.10$	100,000
5,000,000	Users × 10%	$5,000,000 \times 0.10$	500,000

Interpretation

At 1 million registered users, approximately 100,000 users could be active simultaneously during peak periods. At 5 million users, this could rise to 500,000 concurrent users. This level of concurrency would require horizontal application scaling, load balancing, efficient database access, caching, and traffic management.

4. Peak Request Load

Assumption

Each active user generates an average of 5 requests per minute during the peak period.

Formula

Requests per Minute (RPM) = Active Users × 5

Requests per Second (RPS) = RPM ÷ 60

Active Users	Formula	Requests/Minute	Requests/Second
10,000	10,000 × 5	50,000 RPM	833.33 RPS
50,000	50,000 × 5	250,000 RPM	4,166.67 RPS
100,000	100,000 × 5	500,000 RPM	8,333.33 RPS
500,000	500,000 × 5	2,500,000 RPM	41,666.67 RPS

Interpretation

At 500,000 active users, the system could receive approximately 2.5 million requests per minute, or 41,667 requests per second. A single application server would not be an appropriate design at this traffic level. A scalable deployment would require multiple stateless API instances behind a load balancer, caching for frequently accessed data, database scaling, and network traffic management.

5. Personalised Listening and Recommendation Experience

Scalability is also important for Moosic's personalised user experience. As the number of users increases, the system should continue to maintain a personalised listening experience for each user rather than providing the same recommendations to everyone.

Moosic can use user preferences and recent listening behaviour to support personalised recommendations. The recommendation process can consider the user's recent moods, listening activity, preferences, and interactions with songs to identify relevant content.

Moo Bot Curated Playlists

Moo Bot extends this personalised experience by suggesting curated playlists based on the mood patterns observed in the user's recent listening experience.

The processing flow can be represented as:

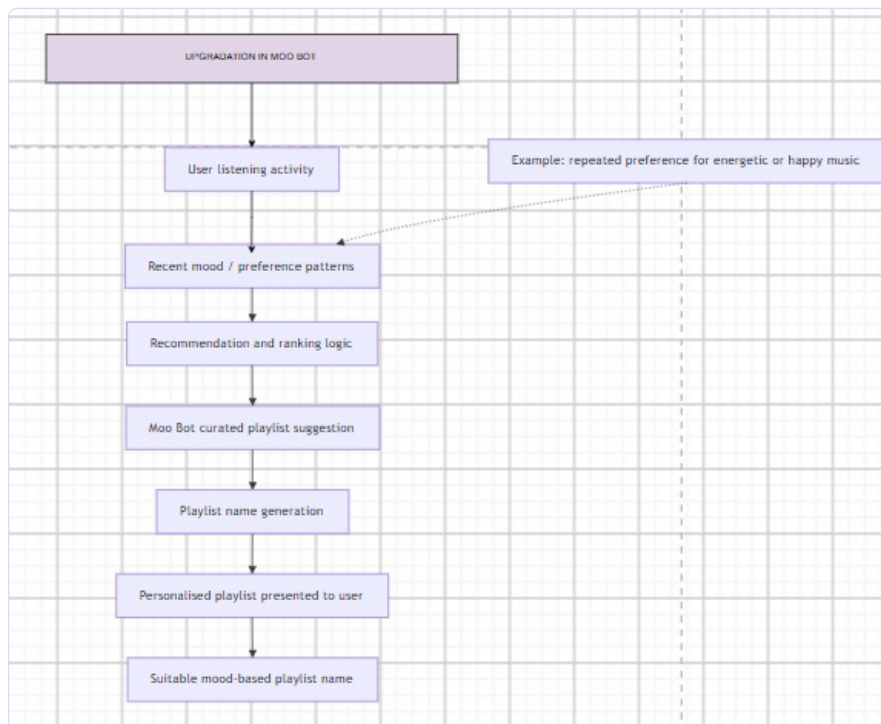


Figure: Moo Bot personalised playlist processing flow

For example, if a user's recent listening behaviour indicates a repeated preference for energetic or happy music, Moo Bot can suggest a curated playlist around that mood and generate a suitable playlist name.

Scaling implication

As the number of users increases from thousands to millions, personalised recommendation processing also increases. The system would therefore need efficient data retrieval, indexed listening-history queries, caching of frequently requested recommendations, and scalable application services. Recommendation processing can be separated from the main request path or handled by background workers at larger scale so that generating personalised playlists does not overload the primary application servers.

This allows Moosic to maintain an individualised listening experience while scaling to a much larger user base.

6. Scaling Implications

Application Scaling

The backend should be deployed as multiple stateless API instances so additional instances can be added as traffic increases. A load balancer can distribute incoming requests across healthy application servers.

Database Scaling

As user activity, playlists, listening history, payments, downloads, and other persistent operations increase, database capacity becomes a major bottleneck. Connection pooling, indexing, query optimisation, caching, and, at larger scale, read replicas or a managed relational database can reduce pressure on the primary database.

Storage Scaling

User-generated and system data will grow with the number of users and interactions. At larger scale, object storage can be used for large files and backups, while the relational database stores structured transactional metadata.

Network Scaling

Higher request rates require sufficient bandwidth, resilient networking, traffic management, and distributed entry points.

A CDN and load balancer can reduce pressure on origin services and improve response times.

Caching

Frequently requested data such as popular songs, mood categories, recommendation results, and dashboard summaries can be cached to reduce repeated database queries.

Monitoring

At scale, monitoring should track request rate, latency, error rate, CPU/memory utilisation, database performance, storage growth, and security events.

Security

Scaling must preserve authentication, authorisation, encrypted connections, security headers, rate limiting, request validation, logging, and controlled access to database and storage resources.

Backup and Recovery

As the amount of persistent data increases, automated backups, tested restoration procedures, and redundant storage become increasingly important.

| 7. Overall Interpretation

The calculations show that Moosic's infrastructure requirements grow significantly as usage increases. The key transition occurs between a small deployment and the 1-million/5-million-user scenarios, where horizontal scaling and distributed infrastructure become necessary. The proposed architecture therefore moves from a single application/database setup toward load-balanced application instances, scalable database infrastructure, caching, distributed storage, stronger monitoring, and resilient backup/recovery mechanisms.